

Comprehensive Guide to Bidirectional RNNs (BRNNs)

Generated Deep Learning Reference

1 Introduction: The Motivation for Bidirectionality

In traditional Recurrent Neural Networks (RNNs) and their variants (like LSTMs and GRUs), the network processes sequence data strictly from left to right. This forward-only approach introduces a critical bottleneck in tasks like Named Entity Recognition (NER).

Consider the two sentences highlighted in your reference material:

1. **Sentence A:** “Dhaval loves *apple*, it keeps him healthy.” (Here, “apple” is a **fruit**).
2. **Sentence B:** “Dhaval loves *apple*, the company produces best electronics.” (Here, “apple” is a **company**).

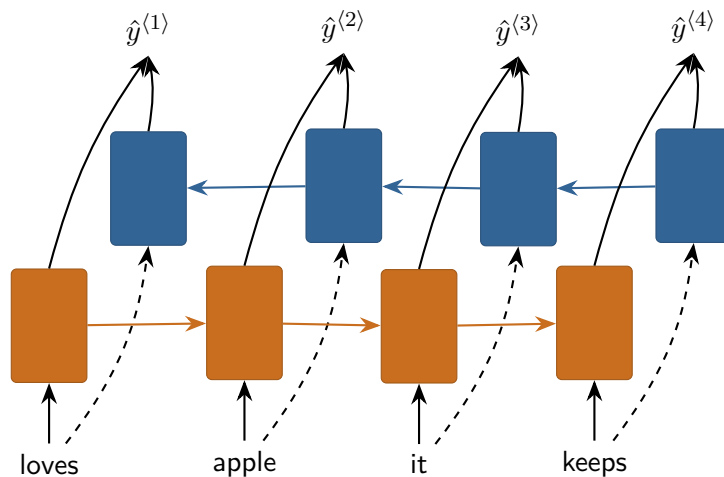
When a standard, unidirectional RNN encounters the word “apple”, it has only seen “Dhaval” and “loves”. It has zero mathematical context from the future words (“healthy” vs. “company”) to make an accurate classification. To resolve this ambiguity, the model must be able to look *ahead* in time.

2 Bidirectional Architecture

A Bidirectional RNN (BRNN) solves this by duplicating the hidden layer. It utilizes two independent processing streams:

- **Forward Layer** ($\vec{h}$): Processes the sequence from time step $t = 1$ to T .
- **Backward Layer** ($\overleftarrow{h}$): Processes the sequence in reverse, from $t = T$ down to 1.

The outputs of both layers are then combined at each time step t to generate the final prediction, allowing the model to have full past and future context simultaneously.



3 Mathematical Formulations

Below are the 10 fundamental equations governing the Bidirectional RNN architecture, extending into Bidirectional GRUs and the backpropagation process.

A. The Core BRNN State Updates

At any given time step t , the network receives input x_t .

1. Forward Hidden State

Calculates the context from the *past* to the present.

$$\vec{h}_t = \tanh(W_{x\vec{h}}x_t + W_{\vec{h}\vec{h}}\vec{h}_{t-1} + b_{\vec{h}}) \quad (1)$$

Where W are weight matrices and b is the bias vector for the forward pass.

2. Backward Hidden State

Calculates the context from the *future* to the present. Note that it relies on h_{t+1} .

$$\overleftarrow{h}_t = \tanh(W_{x\overleftarrow{h}}x_t + W_{\overleftarrow{h}\overleftarrow{h}}\overleftarrow{h}_{t+1} + b_{\overleftarrow{h}}) \quad (2)$$

3. Combined Hidden State (Concatenation)

The two states are typically concatenated to form the complete feature representation for time t .

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (3)$$

B. Output Generation

Once the bidirectional hidden state h_t is constructed, it is mapped to the output space.

4. Pre-activation Output

$$y_t = W_{hy}h_t + b_y \quad (4)$$

Alternatively, if weights are kept separate before addition: $y_t = W_{\vec{h}y}\vec{h}_t + W_{\overleftarrow{h}y}\overleftarrow{h}_t + b_y$

5. Final Output Activation (Softmax)

For classification tasks like Named Entity Recognition, the raw scores are converted to probabilities.

$$\hat{y}_t = \text{softmax}(y_t) = \frac{\exp(y_{t,i})}{\sum_{j=1}^K \exp(y_{t,j})} \quad (5)$$

C. Applying GRUs Bidirectionally

Standard RNNs suffer from vanishing gradients. In modern deep learning, the hidden layers ($\vec{h}$ and $\overleftarrow{h}$) are replaced with GRU (or LSTM) cells.

6. Forward GRU Update Gate

$$\vec{z}_t = \sigma(W_{x\vec{z}}x_t + U_{\vec{z}}\vec{h}_{t-1} + b_{\vec{z}}) \quad (6)$$

7. Backward GRU Update Gate

Notice the backward gate uses h_{t+1} instead of h_{t-1} .

$$\overleftarrow{z}_t = \sigma(W_{x\overleftarrow{z}}x_t + U_{\overleftarrow{z}}\overleftarrow{h}_{t+1} + b_{\overleftarrow{z}}) \quad (7)$$

D. Loss and Optimization

8. Sequence Cross-Entropy Loss

To train the model, we calculate the loss across the entire sequence of length T .

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \sum_{c=1}^C Y_{t,c} \log(\hat{y}_{t,c}) \quad (8)$$

Where $Y_{t,c}$ is the ground truth label (1 if class c is correct, 0 otherwise).

During Backpropagation Through Time (BPTT), gradients are calculated for both directions independently.

9. Output Gradient w.r.t Forward Weights

$$\frac{\partial \mathcal{L}}{\partial W_{\vec{h}_y}} = \sum_{t=1}^T (\hat{y}_t - Y_t) \vec{h}_t^T \quad (9)$$

10. Output Gradient w.r.t Backward Weights

$$\frac{\partial \mathcal{L}}{\partial W_{\overleftarrow{h}_y}} = \sum_{t=1}^T (\hat{y}_t - Y_t) \overleftarrow{h}_t^T \quad (10)$$

4 Summary: Why BRNNs?

Bidirectional networks are incredibly powerful because they map human reading comprehension closely—when we read a sentence, our brain naturally uses the words appearing *after* an ambiguous word to infer its meaning. This is why Bidirectional processing is a staple in advanced models, serving as the direct predecessor to the fully contextualized Self-Attention mechanisms found in Transformers.